Here's a **practical AS/400 → Snowflake Data Validation SQL Toolkit** you can actually drop into your pipelines, DBT tests, or ad-hoc reconciliation work. I'll assume you have:

- Raw AS/400 data landed (e.g. `RAW_AS400.TABLE_X`)
- Final Snowflake target (e.g. `DW.TABLE_X`)
- A stable business key (e.g. `ACCOUNT_ID`)

1. Row count validation

Sql Copy

```sql
-- Row count comparison: AS400 vs Snowflake
SELECT
  'TABLE_X' AS table_name,
  (SELECT COUNT(*) FROM RAW_AS400.TABLE_X) AS as400_count,
  (SELECT COUNT(*) FROM DW.TABLE_X)        AS snowflake_count,
  (SELECT COUNT(*) FROM RAW_AS400.TABLE_X)
    - (SELECT COUNT(*) FROM DW.TABLE_X)    AS count_diff;
```

2. Aggregate metric validation

Sql Copy

```sql
-- Compare key numeric metrics (e.g. BALANCE, PRINCIPAL)
SELECT
  'TABLE_X' AS table_name,
  'BALANCE' AS metric,
  (SELECT SUM(BALANCE) FROM RAW_AS400.TABLE_X) AS as400_sum,
  (SELECT SUM(BALANCE) FROM DW.TABLE_X)        AS snowflake_sum,
  (SELECT SUM(BALANCE) FROM RAW_AS400.TABLE_X)
    - (SELECT SUM(BALANCE) FROM DW.TABLE_X)    AS sum_diff;
```

You can repeat this pattern for `INTEREST`, `FEES`, etc.

3. distinct value validation

Sql Copy

```sql
-- Distinct count comparison for key columns
SELECT
  'TABLE_X' AS table_name,
  'ACCOUNT_ID' AS column_name,
  (SELECT COUNT(DISTINCT ACCOUNT_ID) FROM RAW_AS400.TABLE_X) AS as400_distinct,
  (SELECT COUNT(DISTINCT ACCOUNT_ID) FROM DW.TABLE_X)        AS snowflake_distinct,
  (SELECT COUNT(DISTINCT ACCOUNT_ID) FROM RAW_AS400.TABLE_X)
    - (SELECT COUNT(DISTINCT ACCOUNT_ID) FROM DW.TABLE_X)    AS distinct_diff;
```

4. hash-based row-level reconciliation

4.1 Build a canonical hash on both sides

Sql Copy

```sql
-- AS400 side (after normalization)
CREATE OR REPLACE TABLE VALIDATION.TABLE_X_AS400_HASH AS
SELECT
  ACCOUNT_ID,
  MD5(
    CONCAT(
      COALESCE(TRIM(ACCOUNT_ID), ''),
      '|', COALESCE(TO_CHAR(BALANCE), ''),
      '|', COALESCE(TO_CHAR(PRINCIPAL), ''),
      '|', COALESCE(TO_CHAR(OPEN_DATE), '')
    )
  ) AS row_hash
FROM RAW_AS400.TABLE_X;
```

Sql Copy

```sql
-- Snowflake side
CREATE OR REPLACE TABLE VALIDATION.TABLE_X_SF_HASH AS
SELECT
  ACCOUNT_ID,
  MD5(
    CONCAT(
      COALESCE(TRIM(ACCOUNT_ID), ''),
```

```
      '|', COALESCE(TO_CHAR(BALANCE), ''),
      '|', COALESCE(TO_CHAR(PRINCIPAL), ''),
      '|', COALESCE(TO_CHAR(OPEN_DATE), '')
    )
  ) AS row_hash
FROM DW.TABLE_X;
```

4.2 Find mismatches

Sql Copy

```
-- Rows present in AS400 but missing or different in Snowflake
SELECT a.ACCOUNT_ID
FROM VALIDATION.TABLE_X_AS400_HASH a
LEFT JOIN VALIDATION.TABLE_X_SF_HASH s
  ON a.ACCOUNT_ID = s.ACCOUNT_ID
WHERE s.ACCOUNT_ID IS NULL
   OR a.ROW_HASH <> s.ROW_HASH;
```

5. schema validation (column names & types)

Sql Copy

```
-- Compare column names and data types between two tables
WITH as400_cols AS (
  SELECT
    COLUMN_NAME,
    DATA_TYPE
  FROM INFORMATION_SCHEMA.COLUMNS
  WHERE TABLE_SCHEMA = 'RAW_AS400'
    AND TABLE_NAME   = 'TABLE_X'
),
sf_cols AS (
  SELECT
    COLUMN_NAME,
    DATA_TYPE
  FROM INFORMATION_SCHEMA.COLUMNS
  WHERE TABLE_SCHEMA = 'DW'
    AND TABLE_NAME   = 'TABLE_X'
)
```

```sql
SELECT
  COALESCE(a.COLUMN_NAME, s.COLUMN_NAME) AS column_name,
  a.DATA_TYPE AS as400_type,
  s.DATA_TYPE AS snowflake_type
FROM as400_cols a
FULL OUTER JOIN sf_cols s
  ON a.COLUMN_NAME = s.COLUMN_NAME
WHERE a.DATA_TYPE <> s.DATA_TYPE
   OR a.COLUMN_NAME IS NULL
   OR s.COLUMN_NAME IS NULL;
```

6. null / domain / pattern checks

6.1 null checks

Sql Copy

```sql
-- Null rate comparison for a critical column
SELECT
  'TABLE_X' AS table_name,
  'CUSTOMER_ID' AS column_name,
  (SELECT COUNT(*) FROM RAW_AS400.TABLE_X WHERE CUSTOMER_ID IS NULL) AS as400_nulls,
  (SELECT COUNT(*) FROM DW.TABLE_X        WHERE CUSTOMER_ID IS NULL) AS snowflake_nulls;
```

6.2 domain checks

Sql Copy

```sql
-- Domain check: STATUS must be in a known set
SELECT STATUS, COUNT(*) AS cnt
FROM DW.TABLE_X
WHERE STATUS NOT IN ('ACTIVE', 'CLOSED', 'SUSPENDED')
GROUP BY STATUS;
```

6.3 pattern checks (e.g. account number format)

Sql Copy

```sql
-- Pattern check: ACCOUNT_ID must be 10 digits
SELECT ACCOUNT_ID
FROM DW.TABLE_X
WHERE ACCOUNT_ID NOT RLIKE '^[0-9]{10}$';
```

7. DBT-friendly test patterns

If you're using DBT, you can wrap many of these as tests:

Sql Copy

```sql
-- Example: dbt generic test for row count parity
SELECT
  CASE
    WHEN (SELECT COUNT(*) FROM RAW_AS400.TABLE_X)
       = (SELECT COUNT(*) FROM DW.TABLE_X)
    THEN 0
    ELSE 1
  END AS test_failure;
```

Any non-zero result fails the test.